ENTERPRISE AGENTIC SYSTEMS: FOUNDATIONS

Day 4: Governance, Evaluation & Ops

Compliance, cost, and operations, closing with a governance review the group runs themselves

DAY 4

Today's agenda

- 09:30 - 10:45: Compliance & Risk: PII, data residency, EU AI Act risk classes
- 11:00 - 11:30: FinOps for Agents: cost attribution, model selection
- 11:30 - 12:00: Operating models: who runs which control
- 13:00 - 14:15: AgentOps: trace collection, observability, chaos engineering, guardrails demo
- 14:30 - 15:45: Capstone: governance & risk review, group presentations
- 15:45 - 16:10: Third-party tooling landscape: red teaming, monitoring, assessment
- 16:10 - 16:30: Course wrap-up, link to the Deep Dive week, feedback

1

Compliance & Risk

PII handling, data residency, and mapping use cases to EU AI Act risk classes

COMPLIANCE & RISK

PII and personal-data handling in agentic systems

- Every tier from Day 1's memory model can hold personal or sensitive data: context, session, long-term store, and retrieved documents alike
- An agent that summarises or compacts memory (Day 1) can inadvertently retain or leak PII through that summary
- **Question to bring to every design review: where does PII enter this system, where is it stored, and for how long?**
- This is not a new discipline. It's your existing data-handling obligations, applied to a new kind of processor

COMPLIANCE & RISK

Data residency

- Where does inference actually happen: which region, which provider, which subprocessor?
- Agentic systems often call multiple tools and models in sequence. Each hop is a potential residency boundary crossing
- **A single EU-region setting on your main model doesn't guarantee every tool call, retrieval index, or sub-agent respects the same boundary**
- Managed cloud and on-premises change who can answer that: a managed service moves some answers into the provider's documentation and subprocessor list, self-hosting moves them onto your own team
- Architecture review question: map every hop in this agent's execution. Does data cross a residency boundary at any of them?

D

DISCUSSION

Is a live system already higher-risk than it's currently being treated?

Take 3-5 minutes. Which of Day 1's four memory tiers is hardest to audit for PII today, and why?

EU AI ACT

Mapping a use case to risk class (illustrative)

Risk class	Example from this week	Obligation weight
Unacceptable risk	Not applicable to the scenarios covered this week	Prohibited outright
High risk	The loan-servicing exception queue (Day 2) if it affects credit decisions	Conformity assessment, documentation, human oversight
Limited risk	The customer-facing mortgage agent (Day 3)	Disclosure, transparency requirements
Minimal risk	An internal drafting aid with full human review of every output	No specific obligations beyond general law

?

QUICK POLL

Pull out your Day 2 architecture sketch. Which EU AI Act risk class would you provisionally place it in, and does that match how it's currently being treated internally?

COMPLIANCE & RISK

The rules banks already work under

- EU AI Act: obligations follow risk class; the high-risk dates have been pushed back, so confirm today's dates with CSOB compliance
- DORA (applying since 17 January 2025): ICT third-party risk, with contract terms, a register of every ICT provider, and attention to concentration
- A model API, a parent-provided platform and an MCP server are all ICT services someone has to record
- Czech Act No. 264/2025 on Cybersecurity (in force 1 November 2025, implementing NIS2): how it sits alongside DORA for a bank is a question for CSOB compliance
- **Architecture review question: "which of this agent's dependencies would appear in the register, and who owns each entry?"**

2

FinOps for Agents

Token-level cost attribution and model selection strategies

Why agent cost is harder to attribute

- A single user request can fan out into many model calls: planning, tool calls, retries, sub-agent delegation
- Without per-call tagging, cost shows up as one large, unattributable line item
- Token-level attribution: tag every call with the originating use case, team, and (where relevant) customer request
- **This is also a safety signal, not just a finance one. A sudden cost spike often means an agent is looping or retrying badly**

Model selection strategy

- Not every step needs your most capable, and most expensive, model
- Route by task: a classification or extraction step may run well on a smaller, cheaper model; complex planning may need a stronger one
- Model choice is also a risk decision, not just a cost one. A cheaper model may have different failure characteristics under adversarial input
- **Whatever you choose, evaluate it against the same safety frameworks from Day 3. A smaller model isn't exempt from sandboxing or schema enforcement**

3

Operating Models

Who runs which control: CSOB, the parent organisation or a vendor, and what must exist before go-live

OPERATING MODELS

Who runs which control

- Every control from this week has an owner: CSOB Security, CSOB IT or platform, the parent organisation, or a vendor
- A control you depend on but do not operate still needs a requirement, evidence, and a way to learn when it changes
- Separate must-have (no go-live without it) from nice-to-have (planned, with a date)
- A shared service moves the ownership question; it does not remove it
- **Architecture review question: "for each control, who runs it, who tells us when it breaks, and who signs off?"**

WORKSHOP

Now: fill in the control ownership matrix

- In groups: take your Day 2 architecture sketch
- The worksheet lists this week's eleven controls, including the new AI-BoM and guardrail rows
- For each, mark who runs it today: CSOB Security, CSOB IT or platform, the parent organisation (KBC) or a vendor
- Mark each must-have or nice-to-have for go-live
- Circle two controls where the owner is unclear or sits outside CSOB

4

AgentOps

Trace collection, observability, and chaos engineering for agents

AGENTOPS

Trace collection & observability

- A trace should answer: what did the agent decide, why, what did it call, and what came back, not just what the final answer was
- This is the direct payoff of Day 2's event-driven design discussion. You can't trace what you didn't design to be traceable
- Tools like LangSmith or TruLens exist to capture this at the framework level, named here as examples of a category, not an endorsement of a specific vendor
- **Ops question for any agent going to production: if this makes a wrong decision at 2am, can someone reconstruct why, tomorrow morning?**

D

DISCUSSION

What alert would have caught Day 3's overnight failure by 2am?

Take 3-5 minutes, referring back to the overnight reconciliation case study from Day 3: what specific trace or alert, if it had existed, would have surfaced that outage before morning instead of after?

Chaos engineering for agents

- Deliberately inject failures (a timed-out tool, a malformed retrieval result, a dropped sub-agent) before production does it for you
- This is Day 3's failure-domain material, made into a standing practice rather than a one-off design discussion
- For agentic systems specifically: also inject adversarial inputs, a Day 3 prompt-injection-style test, as part of the same practice
- The goal isn't zero failures. It's confidence that failures are visible, contained, and escalate correctly when they happen

Guardrails: four places a control can sit

- Input: check what goes in: the user message, pasted documents, retrieved text
- Output: check what comes out: personal data, format, claims
- Action: check the tool call before it runs: limits, entitlements, policy
- Human: a person approves what is high-risk or unclear
- Each catches different things; none catches everything

GUARDRAILS

Every guardrail has an error rate

- False negatives: the bad thing gets through
- False positives: legitimate work is blocked
- Cost: latency, money, user frustration
- Bypass: rewording, other languages, indirect injection
- Guardrails are configuration: version them and measure them

G

GUIDED WALKTHROUGH

Guardrails, live

About 25 minutes, presenter-led. Predict before each step: what will this layer catch, and what will get past it?

D

DISCUSSION

Which guardrail would you trust on its own, and what would you want measured before you did?

Take about 5 minutes. Then: who owns tuning it, and who would notice if it quietly stopped working?

CASE STUDY

The vendor's compliance claim

A spare scenario for the capstone if a group has neither its own Day 2 sketch nor a Day 2/3 case study to hand: a vendor claims "full GDPR compliance and built-in EU AI Act alignment" with real gaps underneath.

CAPSTONE

Now: governance & risk review

- In groups: take your Day 2 architecture sketch (or a provided scenario) through a governance review
- Classify its EU AI Act risk tier and justify it
- Identify its PII/data-residency touchpoints across every tier and protocol boundary
- State the operational bar: observability, guardrails and where they sit, configuration record and drift detection, chaos testing; who would operate each, and whether it is must-have

Tooling landscape: five categories

- Automated red teaming: garak, PyRIT, promptfoo, DeepTeam
- Guardrails and runtime protection: Guardrails AI, NeMo Guardrails, Llama Guard, cloud services
- Evaluation and assessment: Inspect, DeepEval, Giskard
- Monitoring and observability: LangSmith, Langfuse, Arize Phoenix, TruLens
- AI security posture and governance: commercial platforms; a consolidating market

TOOLING LANDSCAPE

Automated red teaming: what it can and can't do

- Can: breadth, repeatability, a re-test after every change, a cheap first pass
- Can't: prove safety, understand CSOB's business logic, reach paths it was not pointed at
- Judged by another model? Sample the verdicts by hand
- A clean report is not a sign-off
- Findings need an owner and a route into change control

TOOLING LANDSCAPE

Monitoring and assessment: what you'd want from tooling

- Behaviour baselined before go-live
- Change detected in outputs, tool use, cost and guardrail block rates
- Automatic re-test after a model, prompt or tool change
- Evidence an auditor can read: what, when, who approved
- Alerts that reach a named owner

TOOLING LANDSCAPE

Choosing tooling: questions to ask any vendor

- What does it test or watch, and what is out of scope?
- How do you show that a finding is real, and what is your miss rate?
- Where does our data go, and who can see it?
- What changes when your models or rules update, and how are we told?
- Can it export evidence into our own tools?
- Who owns the findings on our side?

ACTIVITY

Match the requirement to a category

- Pick three requirements from your capstone list
- Match each to a category, or to a person or process
- Write one question you would ask a vendor
- Share one match per group

Course wrap-up

- You now share a vocabulary: autonomy spectrum, memory tiers, agentic RAG, the major architectural patterns and protocols
- You've practised the diagnostic questions that travel: blast radius, boundary identity, escalation specificity, residency per hop
- Guardrails are controls with error rates, and agents are configurations that drift: measure and record both
- Next in the programme: the Agentic Systems Deep Dive week, goes further into the material covered at foundations level this week
- Thank you. Feedback form to follow from Framework Training